

Databases Laboratory 2:

Efe Kardes (2757072)

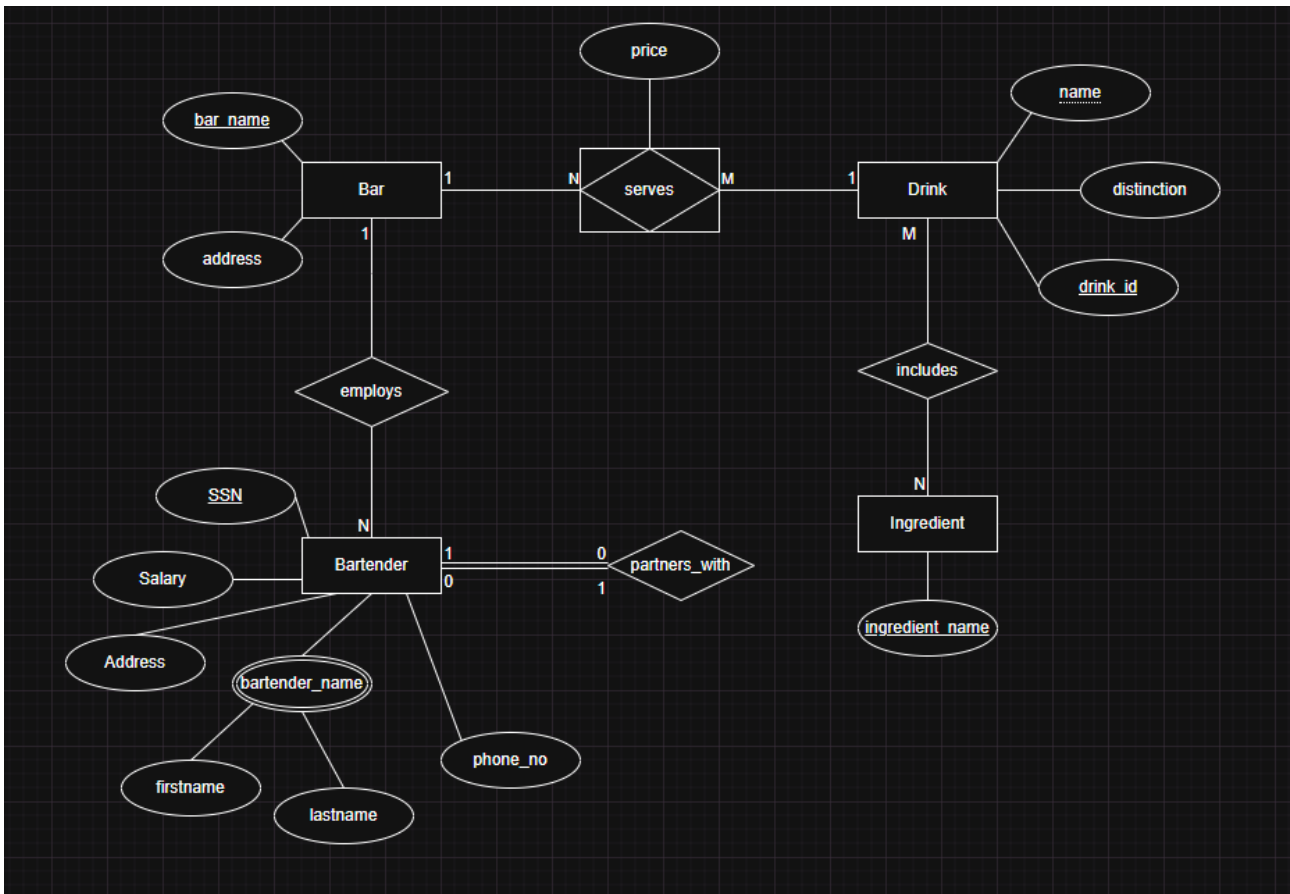
Emir Avni Sahin (2756541)

Gafur Samet Saglik (2667849)

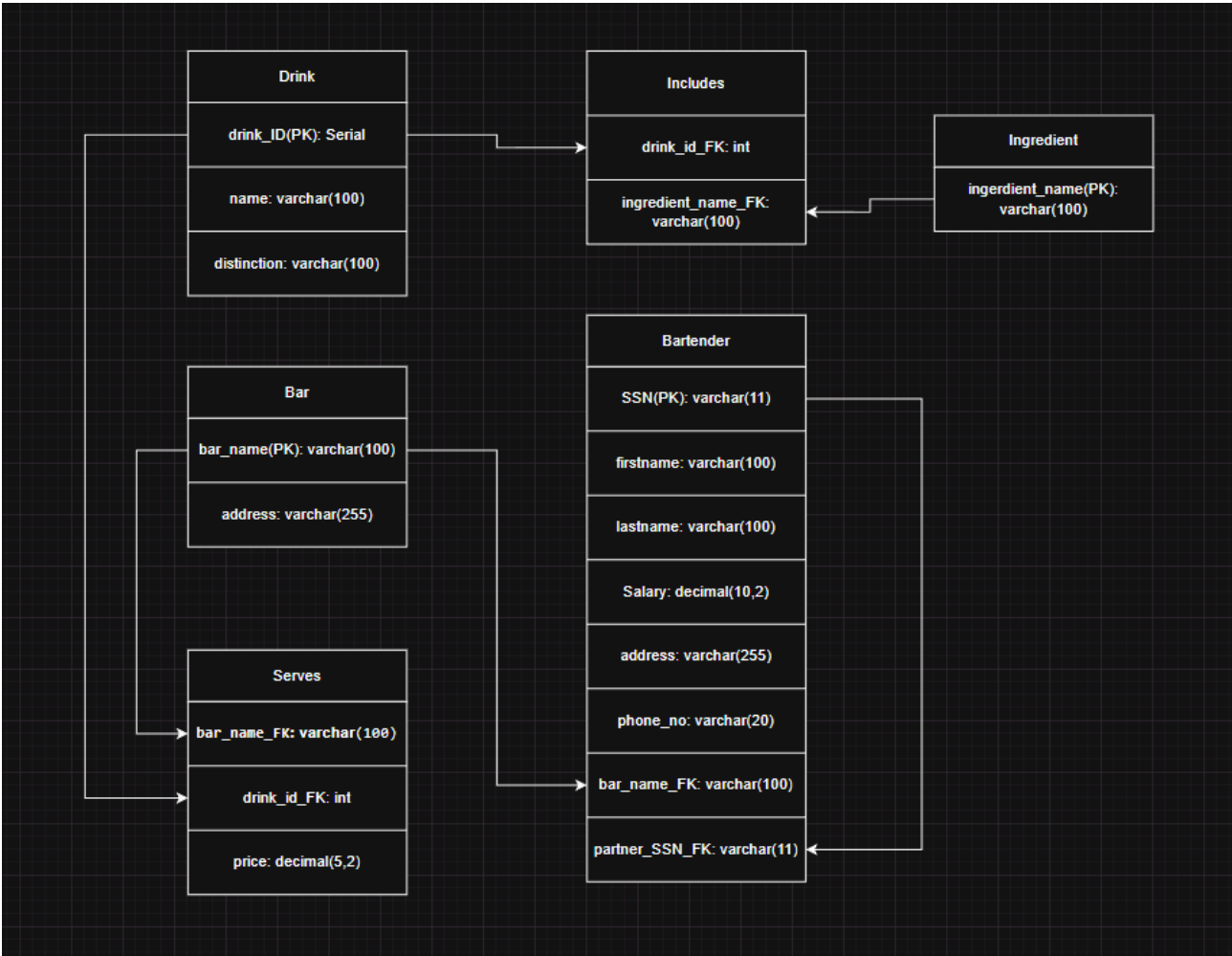
November 8, 2025

Assisgnment 1. Entity-Relationship Model (ERM)

a)ERM



b)RM



c)SQL Statements (DDL & DML)

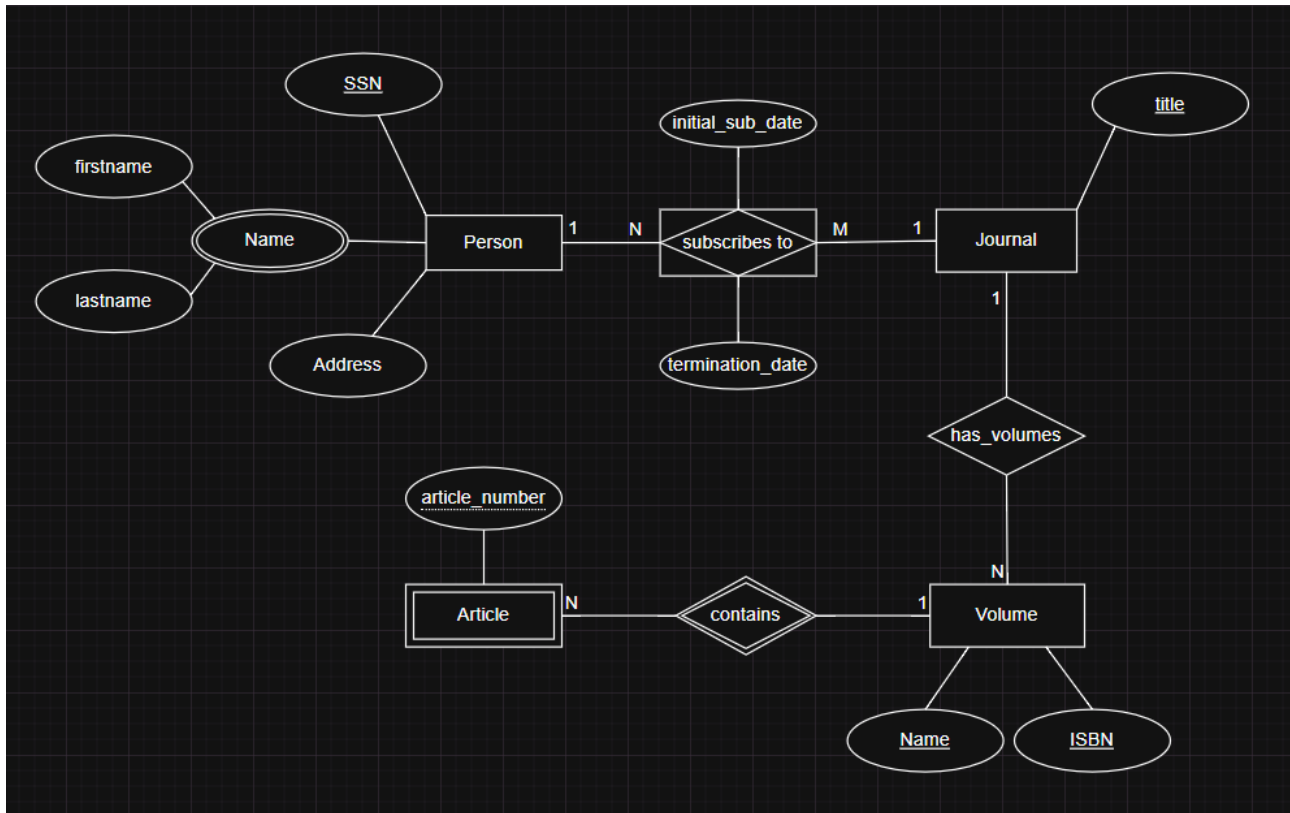
```
1
2 CREATE TABLE Bar (
3     bar_name VARCHAR(100) PRIMARY KEY,
4     address VARCHAR(255) NOT NULL
5 );
6
7 CREATE TABLE Drink (
8     drink_id SERIAL PRIMARY KEY,
9     name VARCHAR(100) NOT NULL,
10    distinction VARCHAR(100)
11 );
12
13 CREATE TABLE Ingredient (
14     ingredient_name VARCHAR(100) PRIMARY KEY
15 );
16
17 -- Dependent tables (with Foreign Keys)
18 CREATE TABLE Bartender (
19     SSN VARCHAR(20) PRIMARY KEY,
20     firstname VARCHAR(100) NOT NULL,
21     lastname VARCHAR(100) NOT NULL,
22     Salary DECIMAL(10, 2) NOT NULL CHECK (Salary > 0),
23     Address VARCHAR(255),
24     phone_no VARCHAR(50),
25
26     -- Implements 1:N employs relationship
27     bar_name_FK VARCHAR(100) NOT NULL,
28     FOREIGN KEY (bar_name_FK) REFERENCES Bar(bar_name),
29
30     -- Implements 0..1 partners_with relationship
31     partner_SSN_FK VARCHAR(20) NULL,
32     FOREIGN KEY (partner_SSN_FK) REFERENCES Bartender(SSN)
33 );
34
35 -- Junction table for M:N serves relationship
36 CREATE TABLE SERVES (
37     bar_name_FK VARCHAR(100) REFERENCES Bar(bar_name),
38     drink_id_FK INT REFERENCES Drink(drink_id),
39     price DECIMAL(5, 2) NOT NULL CHECK (price >= 0),
40
41     -- Composite Primary Key
42     PRIMARY KEY (bar_name_FK, drink_id_FK)
43 );
44
45 -- Junction table for M:N includes relationship
46 CREATE TABLE INCLUDES (
47     drink_id_FK INT REFERENCES Drink(drink_id),
48     ingredient_name_FK VARCHAR(100) REFERENCES Ingredient(ingredient_name),
49
50     -- Composite Primary Key
51     PRIMARY KEY (drink_id_FK, ingredient_name_FK)
52 );
```

DML (INSERT Statements)

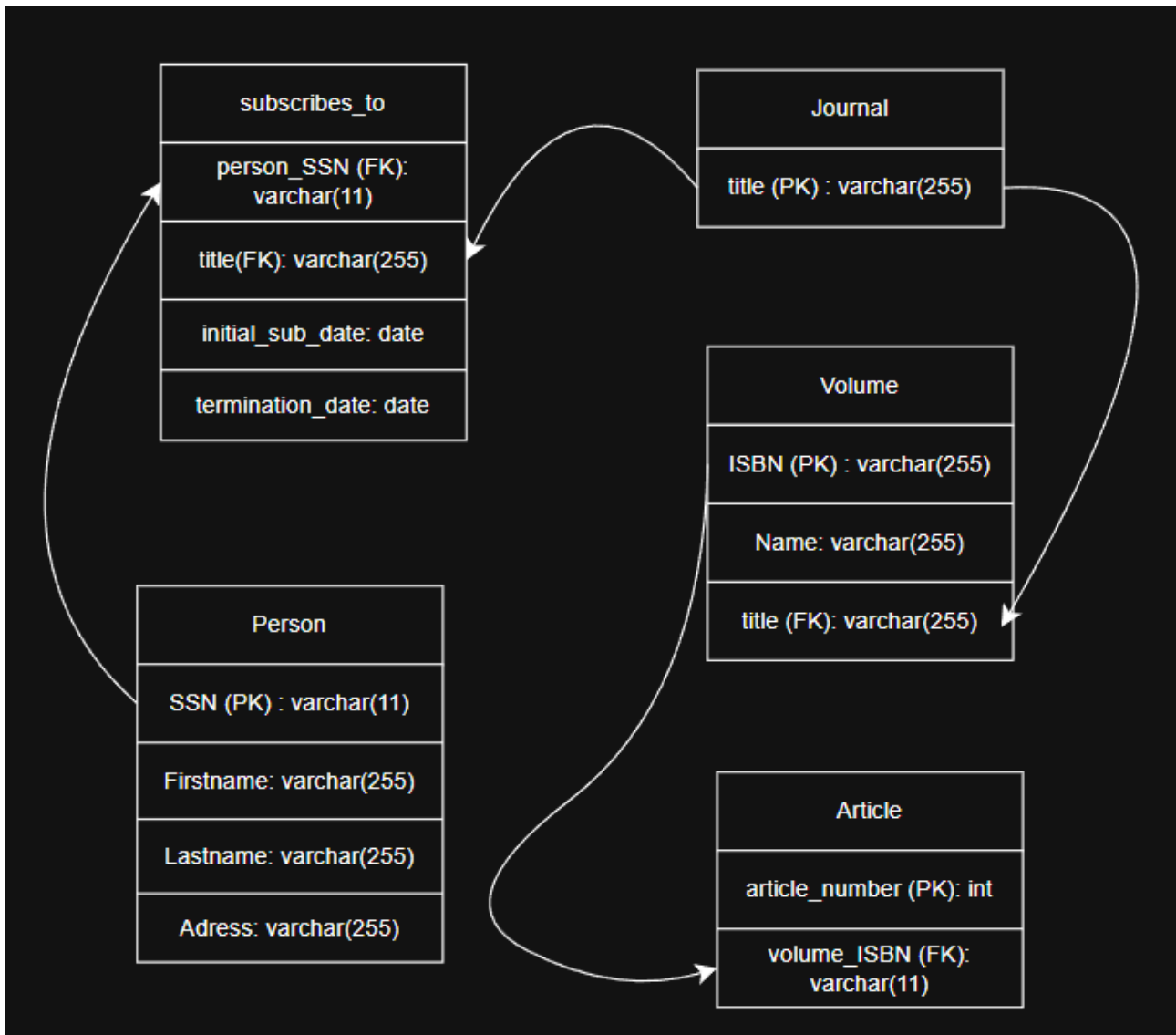
```
1  -- 1. Insert Bars
2  INSERT INTO Bar (bar_name, address) VALUES
3  ('The Salty Dog', '123 Main St'),
4  ('The Topsy Fox', '456 Oak Ave');
5
6  -- 2. Insert Drinks (Drink IDs 1, 2, 3 will be auto-generated)
7  INSERT INTO Drink (name) VALUES
8  ('Margarita'),
9  ('Old Fashioned'),
10 ('Margarita v2'); -- This is the "improved" version
11
12 -- 3. Insert Ingredients
13 INSERT INTO Ingredient (ingredient_name) VALUES
14 ('Tequila'),
15 ('Lime Juice'),
16 ('Cointreau'),
17 ('Bourbon'),
18 ('Sugar Cube'),
19 ('Angostura Bitters');
20
21 -- 4. Insert Bartenders
22 INSERT INTO Bartender (SSN, firstname, lastname, Salary, Address, phone_no,
23   bar_name_FK, partner_SSN_FK) VALUES
24 ('111-22-333', 'Anna', 'Smith', 3000, 'Addr 1', '555-1111', 'The Salty Dog',
25   NULL),
26 ('444-55-666', 'Ben', 'Jones', 3100, 'Addr 2', '555-2222', 'The Salty Dog',
27   NULL),
28 ('777-88-999', 'Carla', 'Brown', 2900, 'Addr 3', '555-3333', 'The Topsy Fox',
29   NULL);
30
31 -- Link partners (handles the optional 1:1)
32 UPDATE Bartender SET partner_SSN_FK = '444-55-666' WHERE SSN = '111-22-333';
33 UPDATE Bartender SET partner_SSN_FK = '111-22-333' WHERE SSN = '444-55-666';
34
35 -- 5. Insert Prices (SERVES)
36 INSERT INTO SERVES (bar_name_FK, drink_id_FK, price) VALUES
37 ('The Salty Dog', 1, 12.50), -- Bar 1, Margarita (ID 1)
38 ('The Topsy Fox', 1, 13.00), -- Bar 2, Margarita (ID 1)
39 ('The Topsy Fox', 2, 15.00); -- Bar 2, Old Fashioned (ID 2)
40
41 -- 6. Insert Recipes (INCLUDES)
42 INSERT INTO INCLUDES (drink_id_FK, ingredient_name_FK) VALUES
43 (1, 'Tequila'), -- Margarita (ID 1)
44 (1, 'Lime Juice'), -- Margarita (ID 1)
45 (1, 'Cointreau'), -- Margarita (ID 1)
46 (2, 'Bourbon'), -- Old Fashioned (ID 2)
47 (2, 'Sugar Cube'), -- Old Fashioned (ID 2)
48 (2, 'Angostura Bitters'); -- Old Fashioned (ID 2)
```

Assignment 2. Journals

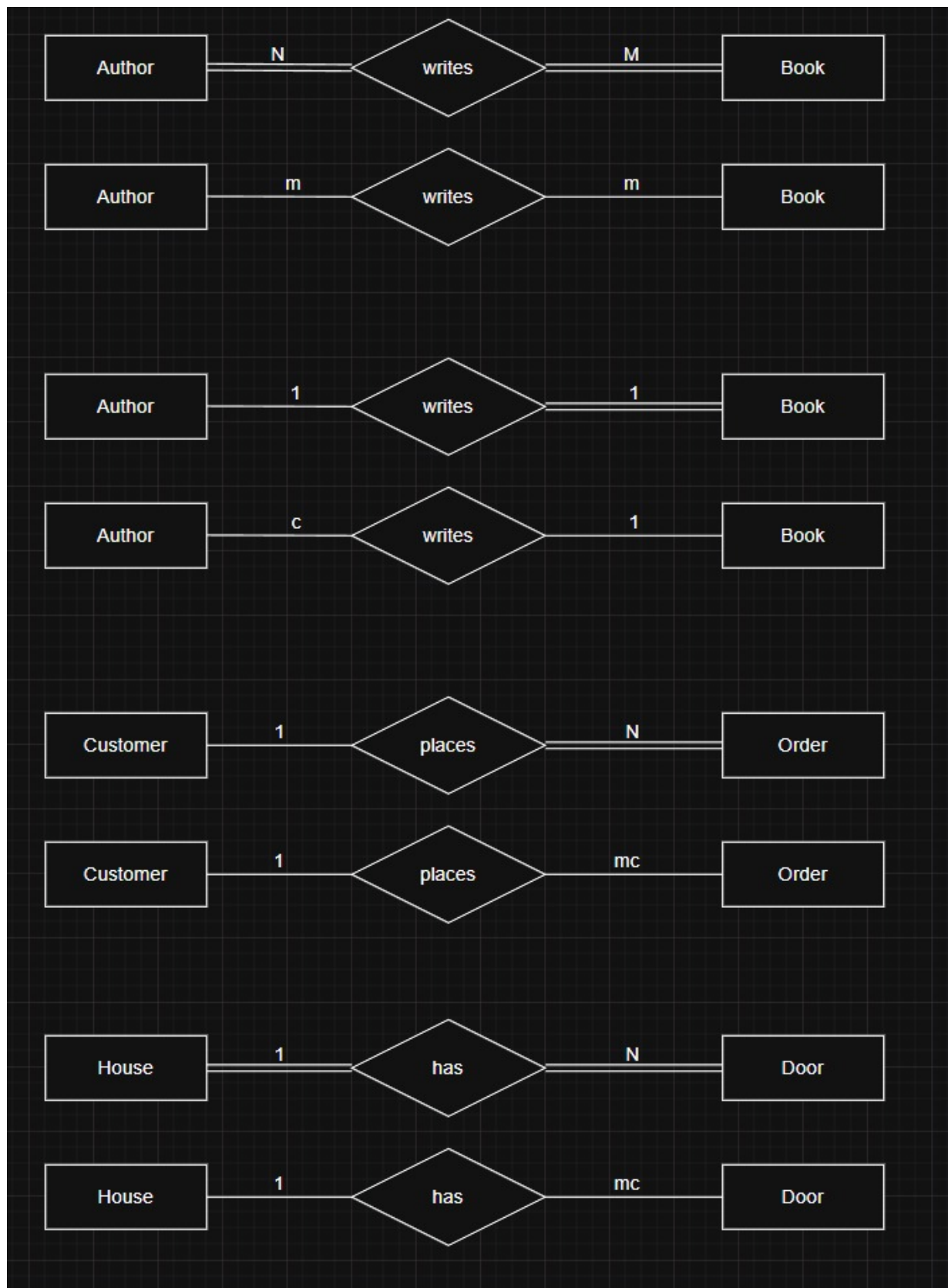
a)ERM



b)RM



Assignment 3: Relationship cardinality notation



Assignment 4: Normalization for a primary school student's list

Student Table			
Student number	Student name	Legal guardian	Class
375428	Yuna Tanaka	Haruko Tanaka, Kenji Tanaka	1A
234982	Lucas Martins	Renata Martins, Paulo Martins	1A
234812	Tariq Hassan	Sara Hassan, Ahmed Hassan	1A
930485	Amira Rahman	Leila Rahman, Hassan Rahman	1B
234980	Emma Johansson	Sofia Johansson	1B
120398	Maya Patel	Priya Patel, Rakesh Patel	1B
234981	Samuel Okafor	Ada Okafor, Chinedu Okafor	4A
239408	Omar Al-Fayed	Lucía Martínez, Carlos Martín	4A
123456	Lucas Martins	Franco Martins	4A

Figure 1: Student Table.

Class Table	
Class teacher	Class
Mr Schneider	1A
Mrs. O'Connor	1B
Mr. Schubert	4A

Figure 2: Class Table.

Enrollment Table		
Student number	Course	Grade
375428	Mathematics 1	A
234982	Mathematics 1	A-
234812	Mathematics 1	B+
375428	Reading 1	A
234982	Reading 1	A
234812	Reading 1	A
930485	Mathematics 1	B+
234980	Mathematics 1	A-
120398	Mathematics 1	B+
930485	Reading 1	B
234980	Reading 1	B+
120398	Reading 1	B-
234981	Mathematics 4	C+
239408	Mathematics 4	A-
234981	English 4	B+
239408	English 4	B-
234981	Art 4	C+
239408	Art 4	A-
123456	English 4	B
123456	Art 4	A
123456	Mathematics 4	B-

Figure 3: Enrollment Table.

Assignment 5: DQL basics for a recipe database

Step 1: Get a table with the names of all recipes

```
SELECT Recipe_Name
```

FROM recipes;

Output:

"Spaghetti Carbonara"
"Chicken Tikka Masala"
"Greek Salad"
"Sushi Rolls"
"Chocolate Chip Cookies"

Step 2: Get the name and cuisine type of all recipes that are easy

```
SELECT Recipe_Name, Cuisine_Type
FROM recipes
WHERE Difficulty = 'Easy';
```

Output:

"Greek Salad" "Mediterranean"
"Chocolate Chip Cookies" "American"

Step 3: Get all ingredients and their amount for Spaghetti Carbonara

```
SELECT i.Ingredient_Name, i.Amount
FROM ingredient2recipe i
JOIN recipes r ON r.Recipe_ID = i.Recipe_ID
WHERE r.Recipe_Name = 'Spaghetti Carbonara';
```

Output:

"Spaghetti" "200 g"
"Eggs" "2"
"Parmesan cheese" "50 g"
"Bacon" "100 g"

Step 4: Get ingredients + amount for all EASY recipes including name + cuisine

```
SELECT r.Recipe_Name,
       r.Cuisine_Type,
       i.Ingredient_Name,
       i.Amount
FROM recipes r
JOIN ingredient2recipe i ON r.Recipe_ID = i.Recipe_ID
WHERE r.Difficulty = 'Easy';
```

Output:

"Greek Salad" "Mediterranean" "Tomato" "2"
"Greek Salad" "Mediterranean" "Cucumber" "1"
"Greek Salad" "Mediterranean" "Feta cheese" "100 g"
"Greek Salad" "Mediterranean" "Olive oil" "2 tbsp"
"Greek Salad" "Mediterranean" "Onion" "2"
"Chocolate Chip Cookies" "American" "Flour" "250 g"
"Chocolate Chip Cookies" "American" "Sugar" "100 g"
"Chocolate Chip Cookies" "American" "Chocolate chips" "80 g"
"Chocolate Chip Cookies" "American" "Butter" "100 g"

Step 5: Get a table of all recipe names that go without onions

```
SELECT r.Recipe_Name
FROM recipes r
WHERE NOT EXISTS (
    SELECT 1
    FROM ingredient2recipe i
    WHERE i.Recipe_ID = r.Recipe_ID
        AND LOWER(i.Ingredient_Name) = 'onion'
);
```

Output:

```
"Spaghetti Carbonara"
"Sushi Rolls"
"Chocolate Chip Cookies"
```

Bonus: Improvement Proposal for table ingredient2recipe

Problem Description:

Currently, ingredient2recipe stores Ingredient_Name as free text. This allows different spellings (e.g. "Onion", "onion", "Onions") which breaks data consistency and makes correct querying harder.

Solution:

add an ingredient table with ingredient_id PK and reference that instead of text